

Relazione Esercitazione 5: Algoritmi di Ordinamento

Matilde Greco

2 maggio 2026

1 Introduzione

In questa esercitazione, ho implementato Mergesort e Quicksort in C++, confrontandoli con gli algoritmi quadratici e con `std::sort`. Lo svolgimento dell'esercitazione si è composto di:

- **Implementazione a partire da pseudo-codici:** La prima parte si è composta dell'implementazione degli pseudo codici di mergesort, quicksort e delle funzioni su cui si basano: la `merge()` e la `partition()`. Tutto ciò è stato aggiunto nel file `sorts.hpp`, copiato dalla precedente esercitazione.
- **Test Unitari:** Oltre ai test già costruiti per l'esercitazione precedente, ossia `test_bubblesort.cpp`, `test_insertionsort.cpp` e `test_selectionsort.cpp`, sono stati aggiunti quelli chiamati allo stesso modo per testare mergesort e quicksort su 100 vettori e sulle stringhe.
- **Misurazione dei tempi:** Nel file `measure_sorts.cpp` è avvenuta la misurazione dei tempi, sono state effettuate misurazioni su 10 vettori di dimensione inferiore a 100. Viene inizializzata una matrice con il comando `vector<vector<int>>()` fatta di 100 righe e 10 colonne. Tale matrice viene riempita tramite il metodo `rf` di `randfiller` con numeri casuali. A seguire si creano copie di tale matrice per ogni test che è necessario fare e si iniziano i test, per ognuno si applica un ciclo `for()`, per calcolare quanto ci impiega il test in questione a ordinare i 100 vettori creati per ogni dimensione. Per ognuno è calcolata la media dei tempi che viene salvata in un vettore `tempi_nome test`, con il metodo `push_back()`. Questi vettori vengono inizializzati all'inizio del codice come vettori dinamici di tipo `double`. Tali medie vengono stampate e a seguire viene definito un `if()` che permette di controllare per quale n_0 gli algoritmi quadratici sono più veloci di quelli logaritmici. Infatti, lo scopo dell'esercitazione è quello di capire fino a che dimensione di vettori conviene usare un algoritmo quadratico e da che dimensione

conviene usarne uno logaritmico. Queste misurazioni vengono salvate in un file di testo chiamato `tempi.txt` che quindi contiene in ordine la dimensione del vettore, il tempo del bubble, quello dell'insertion, del selection, del quicksort, del mergesort e dell'hybrid.

- **Grafico dei tempi:** qui sotto viene riportato il grafico dei tempi, costruito con le misurazioni:

Grafico e Tabella dei tempi

Dim	Bubble	Insertion	Selection	Quick	Merge	Hybrid
0	3e-09	2e-09	3e-09	5e-09	5e-09	2e-09
10	5.06e-07	1.87e-07	4.83e-07	3.81e-07	1.90e-06	2.17e-07
20	2.24e-06	4.68e-07	1.45e-06	9.39e-07	4.09e-06	7.73e-07
30	5.81e-06	6.18e-07	2.61e-06	1.63e-06	6.01e-06	9.68e-07
40	6.64e-06	1.67e-06	4.65e-06	1.84e-06	9.44e-06	1.57e-06
50	8.23e-06	9.01e-07	4.67e-06	2.86e-06	6.02e-06	2.11e-06
60	1.36e-05	3.10e-06	4.23e-06	6.10e-06	5.12e-06	1.73e-06
70	1.75e-05	1.77e-06	4.75e-06	3.43e-06	5.79e-06	2.20e-06
80	1.75e-05	2.30e-06	8.04e-06	4.07e-06	6.93e-06	2.19e-06
90	1.92e-05	1.83e-06	7.36e-06	3.43e-06	7.12e-06	2.61e-06
100	2.27e-05	2.42e-06	8.54e-06	4.28e-06	7.70e-06	2.90e-06

Tabella 1: Confronto dei tempi medi di esecuzione (in secondi).

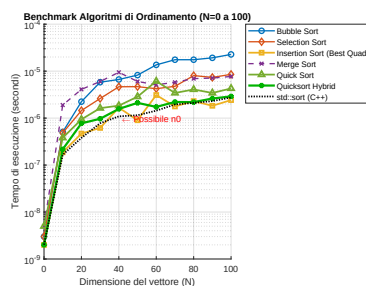


Figura 1: Grafico esportato da MATLAB in formato PDF.

All'interno del grafico si vedono i vari comportamenti degli algoritmi e viene indicato con n_0 il possibile punto in cui diventa più veloce un algoritmo logaritmico rispetto a uno quadratico. In generale l'algoritmo meno efficiente è il Bubblesort, la sua curva è la più alta, ossia risulta essere il più lento, che mostra una rapida crescita nel corso dell'aumentare delle dimensioni. Il Selectionsort si trova in mezzo a livello di prestazioni tra gli algoritmi quadratici. Il più veloce è l'Insertionsort, è estremamente competitivo. Il Mergesort per n piccoli è abbastanza lento, così come il Quicksort che migliora le prestazioni con l'aumentare delle dimensioni. Dalle misurazioni effettuate, si osserva che la soglia n_0 oltre la quale conviene utilizzare algoritmi logaritmici si attesta intorno a $n_0 = 40$.

- **Quicksort Hybrid:** dal grafico era evidente notare come per vettori di piccole dimensioni fosse più efficiente l'Insertion Sort tra gli algoritmi quadratici. Per migliorare ulteriormente le prestazioni si è implementato un secondo algoritmo sotto il nome di Quick Sort Hybrid in `sorts.hpp`. In questo file sono stati implementati: una nuova funzione `partition`, un nuovo `insertion`, designati ad hoc per l'Hybrid e un algoritmo chiamato `quick_sort_hybrid()`. All'interno di questo nuovo algoritmo la ricorsione tipica del quicksort non viene interrotta, come nel tradizionale, quando la dimensione del vettore diventa uno, ma quando essa scende al di sotto della soglia n_0 . A tal punto il sotto-vettore viene ordinato tramite l'Insertion sort implementato appositamente. Come è visibile dal grafico, la curva dell'hybrid si sovrappone a quella dell'Insertion per dimensioni minori di n_0 , questo indica che funziona correttamente, ossia che delega l'ordinamento di un vettore per quelle dimensioni all'algoritmo più veloce. Dopo la soglia, continua a seguire la crescita logaritmica tipica degli algoritmi tipo quicksort. Generalmente la sua linea è più bassa rispetto a quella del quicksort tradizionale, questo significa che a livello di ottimizzazione è migliore.

2 Conclusioni

L'esperimento ha dimostrato che la teoria della complessità computazionale si deve sempre relazionare alla realtà progettuale. Sebbene algoritmi come il Mergesort siano più potenti per gestire milioni di dati, questi hanno dei costi che li rallentano su liste piccole, come un uso della memoria extra, piuttosto che gestione di calcoli difficili. Per questo motivo, se bisogna ordinare vettori di pochi elementi, un algoritmo più semplice e 'leggero' come l'Insertion Sort risulta più veloce ed efficiente.